Logistiq: Adding Supplier Selection & Product Editing

A build-it-yourself guide

August 14, 2026

Overview

This guide walks through two features for the Products area of Logistiq:

1. **Selecting a supplier when creating a new product** — so a product can be linked to who you buy it from, right from the "New product" form.
2. **Editing product info from the product detail page** — right now, the detail page (`app/dashboard/inventory/products/[id]/page.tsx`) is read-only.

Both features follow patterns already used elsewhere in the codebase (`orchestrate/register` actions, the `ACTION_ROLES` permission map, and the modal-form components under `app/components/dashboard/Inventory/products/`), so the fastest way to build these is to copy the shape of what already exists for **categories** and **create-product**, rather than inventing a new pattern.

No code in this guide has been written into your project — it's a map of what to change and where, with reference snippets you can adapt.

Part 1 — Supplier selection on product creation

1.1 What already exists

Good news: most of the plumbing is already there.

- `app/dashboard/inventory/products/page.tsx` already calls `orchestrate({ action: "listSuppliers" }, ctx)` and has a `suppliers` array in scope.
- That `suppliers` array is already passed down to `ProductsTable`, which uses it for the **Reorder** modal.
- It is **not** currently passed into `CreateProductModal` — that's the main gap on the frontend.

The bigger gap is on the data model: `Product` has no relationship to `Supplier` at all right now. `Supplier` only connects to `PurchaseOrder`. So before touching any UI, you need to decide what "supplier" means on a product, then add that relationship.

1.2 Decide what the field means

The natural choice, and the one this guide assumes, is a **preferred/default supplier** — an optional field on `Product` that says "when I need to restock this, this is who I usually buy it from." It does not replace the supplier picked on an individual purchase order; it's just a default that could later pre-fill the Reorder flow.

1.3 Schema change

In `prisma/schema.prisma`, add an optional relation to the `Product` model:

```
model Product {
  id String @id @default(cuid())

  organizationId String
  organization Organization @relation(references: [id], fields: [organizationId])

  sku String
  name String
  reorderPoint Int @default(0)
  attributes Json

  categoryId String?
  category Category? @relation(fields: [categoryId], references: [id])

  supplierId String?
  supplier   Supplier? @relation(fields: [supplierId], references: [id])

  price Decimal? @db.Decimal(10,2)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@unique([organizationId, sku])
  inventoryItems InventoryItem[]
  inventoryRecords InventoryRecord[]
  purchaseOrderLines PurchaseOrderLine[]
  salesOrderLines SalesOrderLine[]
}
```

And give `Supplier` the reverse side of the relation (Prisma requires both sides to be declared):

```
model Supplier {
```

```
  id String @id @default(cuid())

  organizationId String
  organization Organization @relation(fields: [organizationId], references: [id])

  name String
  email String?
  phone String?

  createdAt DateTime @default(now())

  purchaseOrders PurchaseOrder[]
  products       Product[]
}
```

1.4 Run the migration

From your own machine (not this sandbox — Prisma's CLI can't reach its engine binaries or your Neon database from here):

```
npx prisma migrate dev --name add_product_supplier
```

This generates a migration file under `prisma/migrations/` and regenerates the Prisma client. Once it's run, the mounted project folder will pick up the change and `prisma.product` calls will know about `supplierId`/`supplier`.

1.5 Update the backend actions

In `app/modules/products/product.ts`:

- **`createProduct`** — accept `supplierId` from the request body and pass it through, the same way `categoryId` is handled:

```
register("createProduct", async (data, ctx) => {
    const product = await prisma.product.create({
        data: {
            organizationId: ctx.organizationId,
            sku: data.sku,
            name: data.name,
            reorderPoint: data.reorderPoint ?? 0,
            attributes: data.attributes ?? {},
            categoryId: data.categoryId ?? null,
            supplierId: data.supplierId ?? null,
            price: data.price ?? null,
        },
        include: {
            category: { select: { id: true, name: true } },
            supplier: { select: { id: true, name: true } },
        },
    });
    return { status: 201, body: { product } };
});
```

- **`listProducts`** and **`getProduct`** — add `supplier: { select: { id: true, name: true } }` next to the existing `category` include, so the frontend gets supplier name/id back without a second request.

No changes are needed in `app/lib/permission.ts` — `createProduct` is already gated to `ADMIN/MANAGER`, and you're just adding a field to an existing action, not a new action.

1.6 Pass suppliers into the create-product form

In `app/dashboard/inventory/products/page.tsx`, `suppliers` is already fetched — just forward it, same as `categories`:

```
<ProductsTable
  products={products}
  suppliers={suppliers}
  categories={categories}
  canReorder={canReorder}
  canCreateProduct={canCreateProduct}
/>
```

(That line is already correct today — nothing to change here.)

In `app/components/dashboard/Inventory/products/ProductsTable.tsx`, thread `suppliers` into the modal it renders:

```
{showCreateProduct && (
  <CreateProductModal
    categories={categories}
    suppliers={suppliers}
    onClose={() => setShowCreateProduct(false)}
  />
)}
```

1.7 Add the field to `CreateProductModal.tsx`

This component already has a working example of exactly this pattern for `category` — a `<select>` bound to a `categoryId` state variable. Mirror it for supplier:

1. Add a `Supplier` type and a `suppliers` prop, same shape as `categories`.
2. Add `const [supplierId, setSupplierId] = useState("")`.
3. Add a `<select>` block right after the category block:

```
<div className="flex flex-col gap-1.5">
  <label className={labelClass}>Preferred supplier (optional)</label>
  <select
    value={supplierId}
    onChange={(e) => setSupplierId(e.target.value)}
    className={`${inputClass} bg-white`}
  >
    <option value="">No preferred supplier</option>
    {suppliers.map((s) => (
      <option key={s.id} value={s.id}>
        {s.name}
      </option>
    ))}
  </select>
</div>
```

4. In `handleSubmit`, add `supplierId: supplierId || undefined` to the JSON body sent to `/api/requests` with `action: "createProduct"`.

Unlike category, there's no "create a new supplier inline" flow to reuse — `createSupplier` exists as an action, but wiring an inline creator is optional polish, not required for the core feature.

1.8 Optional: show the supplier elsewhere

Once the field exists, you'll likely also want it visible where category already shows up:

- `ProductsTable.tsx` — add a "Supplier" column next to "Category," reading `p.supplier?.name`.
- `app/dashboard/inventory/products/page.tsx` — include `supplier` in the `RawProduct` type and the mapped `products` array (same treatment as `category`).
- The product detail page — see Part 2, since editing and displaying land in the same place.

Part 2 — Editing a product from its detail page

2.1 Decide what's editable

Reasonable first cut, matching what `CreateProductModal` already lets you set: `name`, `sku`, `reorderPoint`, `price`, `categoryId`, `supplierId` (once Part 1 exists), and `attributes` (description, image URL, dimensions, cost, notes, custom fields).

2.2 Add an `updateProduct` backend action

In `app/modules/products/product.ts`, add a new registered action. Follow `getProduct`'s pattern for verifying the product belongs to the caller's org before touching it:

```
register("updateProduct", async (data, ctx) => {
    const { productId } = data;

    if (!productId) {
        return { status: 400, body: { error: "productId is required" } };
    }

    const existing = await prisma.product.findFirst({
        where: { id: productId, organizationId: ctx.organizationId },
    });

    if (!existing) {
        return { status: 404, body: { error: "Product not found" } };
    }

    const product = await prisma.product.update({
        where: { id: productId },
        data: {
            sku: data.sku ?? existing.sku,
            name: data.name ?? existing.name,
            reorderPoint: data.reorderPoint ?? existing.reorderPoint,
            price: data.price ?? existing.price,
            categoryId: data.categoryId ?? null,
            supplierId: data.supplierId ?? null,
            attributes: data.attributes ?? existing.attributes,
        },
        include: {
            category: { select: { id: true, name: true } },
            supplier: { select: { id: true, name: true } },
        },
    });

    return { status: 200, body: { product } };
});
```

A couple of things worth double-checking as you write this:

- The `findFirst` scoped by `organizationId` is what stops one org from editing another org's product by guessing an id — don't skip it.
- Decide deliberately whether `categoryId ?? null`/`supplierId ?? null` is what you want (clearing the field when omitted) versus `?? existing.categoryId` (leaving it alone when omitted). It depends on whether your edit form always sends the full current value or only changed fields.

2.3 Add a permission entry

In `app/lib/permission.ts`, add `updateProduct` next to `createProduct` in the Products section:

```
createProduct: [...ALWAYS_ALLOWED],
updateProduct: [...ALWAYS_ALLOWED],
```

(Adjust the role list if you want, say, `PURCHASING` to be able to edit price/supplier without being a full `MANAGER` — but `ADMIN`/`MANAGER`-only matches how creation is already locked down.)

2.4 Build an `EditProductModal` component

Create `app/components/dashboard/Inventory/products/EditProductModal.tsx` as a sibling to `CreateProductModal.tsx`. The fastest path is to **copy `CreateProductModal.tsx` and adapt it**, rather than starting from scratch:

- Accept a `product` prop (the full record — id, sku, name, reorderPoint, price, categoryId, supplierId, attributes) instead of starting from empty strings, and use those values as the `useState` initial values.
- Change the submit action from `"createProduct"` to `"updateProduct"`, and include `productId: product.id` in the body.
- Change the button/header text ("Edit product" / "Save changes").
- The "more details" fields (description, barcode, dimensions, cost, notes, custom attributes) need to be **parsed back out** of `product.attributes` into individual state variables when the modal opens, so editing doesn't wipe out fields the form doesn't have a dedicated input for. The detail page already has this exact parsing logic (`isAttributeRecord`, `RESERVED_ATTRIBUTE_KEYS`, and the block that reads `attrs.description`, `attrs.imageUrl`, etc.) in `app/dashboard/inventory/products/[id]/page.tsx` — reuse or share that logic rather than rewriting it.

2.5 Wire an "Edit" button into the detail page

`app/dashboard/inventory/products/[id]/page.tsx` is a **server component**, so the new modal needs to live behind a small client wrapper, the same way `ProductDetailActions.tsx` wraps `ReorderModal`. Two options:

- **Simplest:** extend `ProductDetailActions.tsx` to also render an "Edit" button and manage a second `open` state for `EditProductModal`, since it already receives `product` and is already a client component next to the Reorder button.
- **Cleaner separation:** create a new `ProductEditAction.tsx` client component, following the same shape as `ProductDetailActions.tsx`, and render it alongside the existing actions in the page's action bar.

Either way, the page currently has disabled placeholder buttons for **Attachments**, **Copy**, and **Deactivate** (search for `title="Not implemented yet"` in `[id]/page.tsx`). "Edit" is a good candidate to sit in that same button row, and — unlike those three — it doesn't need a new feature underneath it since `updateProduct` will exist after Part 2.

You'll need to pass the full raw product data (not just `{id, name, sku}` like `ProductDetailActions` currently receives) down to whichever component renders the Edit button, since the edit form needs every current value to pre-fill itself.

2.6 Refresh after saving

Match the existing pattern from `CreateProductModal.tsx` and `ReorderModal.tsx`: on a successful `fetch` to `/api/requests`, call `router.refresh()` and then close the modal. Because `page.tsx` is a server component that re-fetches on every navigation/refresh, this is enough to show the updated values immediately — no separate client-side state syncing needed.

Verification checklist

Once both parts are wired up:

1. **Type-check:** `npx tsc --noEmit` — will catch prop-shape mismatches (e.g. a component still expecting `categories` but not receiving `suppliers`) immediately.
2. **Lint:** `npx eslint app/components/dashboard/Inventory/products app/modules/products app/dashboard/inventory/products` — the codebase is currently clean, so any new warning is something you introduced.
3. **Manual test — creation:** Open "New product," confirm the supplier dropdown lists your org's suppliers, submit with and without one selected, and confirm the created product's `supplierId` is set (or `null`) as expected.
4. **Manual test — editing:** Open a product's detail page, edit a field (start with something low-risk like `name` or `price`), save, and confirm the page reflects the change after `router.refresh()` without a manual reload.
5. **Cross-org check:** Try hitting `updateProduct` with a `productId` that belongs to a different organization (e.g. via browser devtools) and confirm you get a 404, not a successful update — this is what the `findFirst` org-scoping check in step 2.2 is protecting against.